

LabTrack

API Documentation

Laboratory Incident Management & AI Assistant — REST API

1. Introduction

LabTrack exposes a single REST API, built with ASP.NET Core 8 and documented automatically via Swashbuckle.AspNetCore (Swagger / OpenAPI). Every endpoint listed in this document is live, interactive, and testable directly from the Swagger UI while the backend is running.

The API is organized into six functional areas, each covered in its own section below:

- Authentication — account creation and login (JWT issuance)
- Incidents — the core incident-tracking CRUD and status workflow
- Documents — knowledge-base upload and the approval governance workflow
- AI Assistant — the Retrieval-Augmented Generation (RAG) question-answering and incident-summary endpoints
- Administration — user account management (Administrator-only)
- Dashboard — aggregate counts and chart data for the frontend dashboard

2. How to Use This API

2.1 Base URL

Local development: <http://localhost:5000/api>

2.2 Content type

All request and response bodies are JSON (Content-Type: application/json), with one exception: document upload, which uses multipart/form-data because it transmits a file.

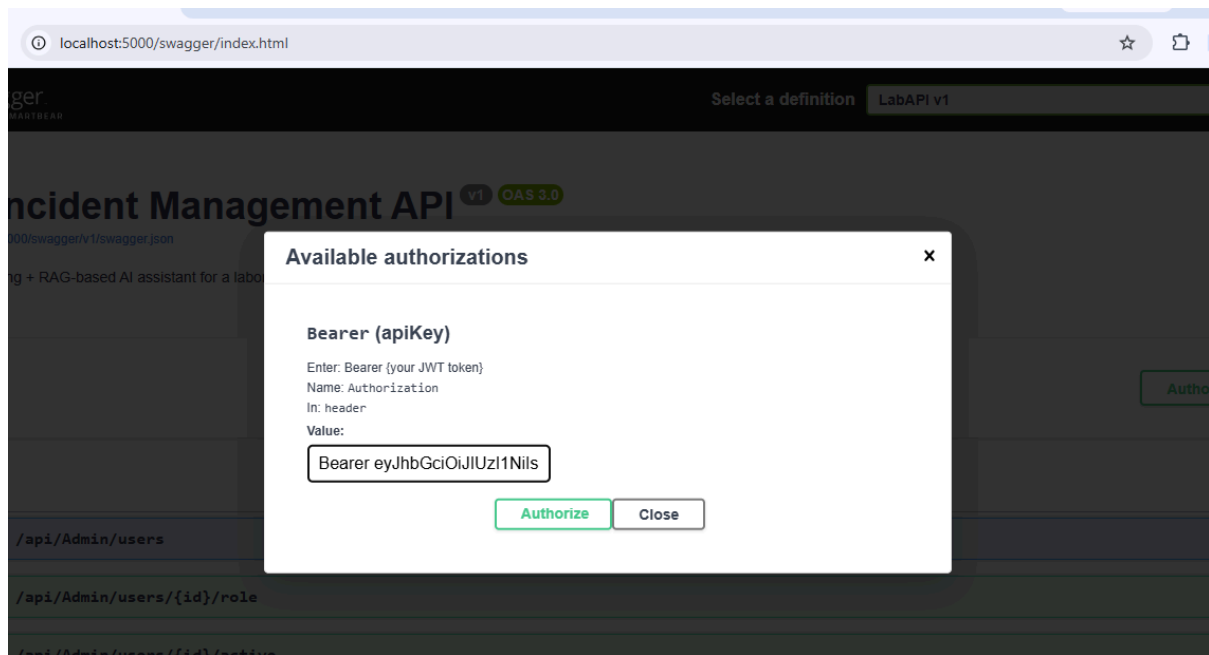
2.3 Enum values

Enum fields (Severity, Status, Role, DocumentStatus) are always sent and received as their string name — for example "Investigating", never a numeric index like 1. This is enforced via a JsonStringEnumConverter applied to every enum type in the codebase.

2.4 Authentication flow

1. Call POST /api/auth/login (or /register) to obtain a JWT token.
2. Attach it to every subsequent request as a header: Authorization: Bearer <token>
3. The token encodes the user's id, name, email, and role; the backend checks the role automatically on any endpoint marked [Authorize(Roles = "...")].

In Swagger UI specifically: click the “Authorize” button (top right, padlock icon), paste Bearer <token>, and every subsequent “Try it out” call in that browser session will include it automatically.



3. Authentication Endpoints

Controller: AuthController — Base route: /api/auth — No authentication required for either endpoint (they are how a token is obtained in the first place).

Method	Route	Auth Required	Description
POST	/api/auth/register	None	Create a new account
POST	/api/auth/login	None	Authenticate and receive a JWT Token

Authorization: *None*
(*public*)

Creates a new user account. New accounts always start with the LabUser role — promotion to Reviewer or Administrator must be done afterward by an existing Administrator via the Admin endpoints.

HOW IT WORKS

- Checks the Users table for an existing account with the same email; rejects with 400 if one exists.
- Hashes the supplied password with BCrypt (BCrypt.Net.BCrypt.HashPassword) — the plain password is never stored.
- Inserts the new User row with Role = LabUser and IsActive = true.
- Immediately generates and returns a JWT, so the new user is logged in right after registering — no separate login step required.

SUCCESSFUL LOGIN CASE

[illegible]

FAILED LOGIN CASE

Responses

Curl

```
curl -X 'POST' \
  'http://localhost:5000/api/Auth/login' \
  -H 'accept: text/plain' \
  -H 'Content-Type: application/json' \
  -d '{
    "email": "angena@gmail.com",
    "password": "Angena12!"
  }'
```

Request URL

```
http://localhost:5000/api/Auth/login
```

Server response

Code	Details
403 <i>Undocumented</i>	Error: Forbidden Response body <pre>{ "Message": "Invalid email or password.", "Details": null }</pre> Response headers <pre>content-type: application/json date: Tue, 01 Sep 2026 09:45:34 GMT server: Kestrel transfer-encoding: chunked</pre>

4. Incident Management Endpoint

Method	Route	Auth Req	Description
GET	/api/incidents	Any logged-in user	Search/filter/sort/paginate incidents
GET	/api/incidents/{id}	Any logged-in user	Get one incident with notes+corrective actions
POST	/api/incidents	Any logged-in user	Create a new incident
PUT	/api/incidents/{id}	Any logged-in user	Update an incident's editable fields
DELETE	/api/incidents/{id}	Administrator	Permanently delete an incident
PATCH	/api/incidents/{id}/status	Reviewer, Administrator	Move to the next workflow status
PATCH	/api/incidents/{id}	Reviewer,	Assign the incident to a

	/assign	Administrator	user
PATCH	/api/incidents/{id}/resolve	Reviewer, Administrator	Mark investigating -> Resolved
POST	/api/incidents/notes	Any logged-in user	Add an investigation note
POST	/api/incidents/{id}/corrective-actions	Any logged-in user	Add a corrective Action

GET /api/incidents

Authorization: *JWT required*

Returns a paginated, filterable, sortable list of incidents. This is the endpoint behind the Incidents list screen and its search/filter bar.

HOW IT WORKS

- Query parameters (all optional): status, severity, category, equipment, fromDate, toDate, search (matches title/description), sortBy, descending, page, pageSize.
- Filters are applied as EF Core LINQ Where clauses, translated directly into a single SQL query — filtering happens in SQL Server, not in application memory.
- Results are wrapped in a PagedResult<T> containing the page of items plus totalCount, page, pageSize, and totalPages, so the frontend can render pagination controls.

Incidents

GET

/api/Incidents

Parameters

Name	Description
Status string (query)	Resolved
Severity string (query)	High
Category string (query)	Category
Equipment string (query)	Equipment
FromDate string(\$date-time) (query)	FromDate
ToDate	

200	Response body <pre>{ "items": [{ "id": 1, "title": "Centrifuge overheating during run", "description": "Centrifuge #3 started emitting a burning smell mid-run and was shut down immediately.", "category": "Equipment Failure", "severity": "High", "status": "Resolved", "equipment": "Centrifuge #3", "occurredAtUtc": "2026-08-14T05:01:58.3828286", "createdByName": "Leo LabUser", "assignedToName": null, "createdAtUtc": "2026-08-16T05:01:58.3823196", "updatedAtUtc": "2026-08-17T04:20:41.3512958", "resolutionDetails": null, "investigationNotes": [], "correctiveActions": [] }], "totalCount": 1, "page": 1, "pageSize": 20, "totalPages": 1 }</pre> Response headers <pre>content-type: application/json; charset=utf-8 date: Tue, 01 Sep 2026 12:24:51 GMT server: Kestrel transfer-encoding: chunked</pre>
Responses	
Code	Description
200	OK

POST /api/incidents

Authorization: *JWT required*

Creates a new incident, always starting in the Open status.

HOW IT WORKS

- Validates the request body against the CreateIncidentRequest DTO (required fields, max lengths) using ASP.NET Core's built-in model validation — an invalid body never reaches the business logic at all.
- Sets Status = Open and CreatedByUserId = the current authenticated user (read from the JWT claims), regardless of what the client sends.
- Writes an initial StatusHistory row ("Incident created") at the same time, so every incident has an audit trail from the moment it exists.

POST

/api/Incidents

Parameters

No parameters

Request body

application/json

```
{
  "title": "Broken glass injury",
  "description": "bleeding and minor cuts leading to wounds or hidden shrads",
  "category": "Slip/Fall",
  "severity": "medium",
  "equipment": "Beaker#B6",
  "occurredAtUtc": "2026-09-01T14:20:29.958Z"
}
```

Execute

Clear

http://localhost:5000/api/Incidents

Server response

Code

Details

201

Undocumented

Response body

```
{
  "id": 3,
  "title": "Broken glass injury",
  "description": "bleeding and minor cuts leading to wounds or hidden shrads",
  "category": "Slip/Fall",
  "severity": "Medium",
  "status": "Open",
  "equipment": "Beaker#B6",
  "occurredAtUtc": "2026-09-01T14:20:29.958Z",
  "createdByName": "Riya Reviewer",
  "assignedToName": null,
  "createdAtUtc": "2026-09-01T14:26:48.4609029Z",
  "updatedAtUtc": "2026-09-01T14:26:48.4609035Z",
  "resolutionDetails": null,
  "investigationNotes": [],
  "correctiveActions": []
}
```

Response headers

```
content-type: application/json; charset=utf-8
date: Tue, 01 Sep 2026 14:26:49 GMT
location: http://localhost:5000/api/Incidents/3
server: Kestrel
transfer-encoding: chunked
```

Responses

Code

Description

200

OK

PATCH /api/incidents/{id}/status

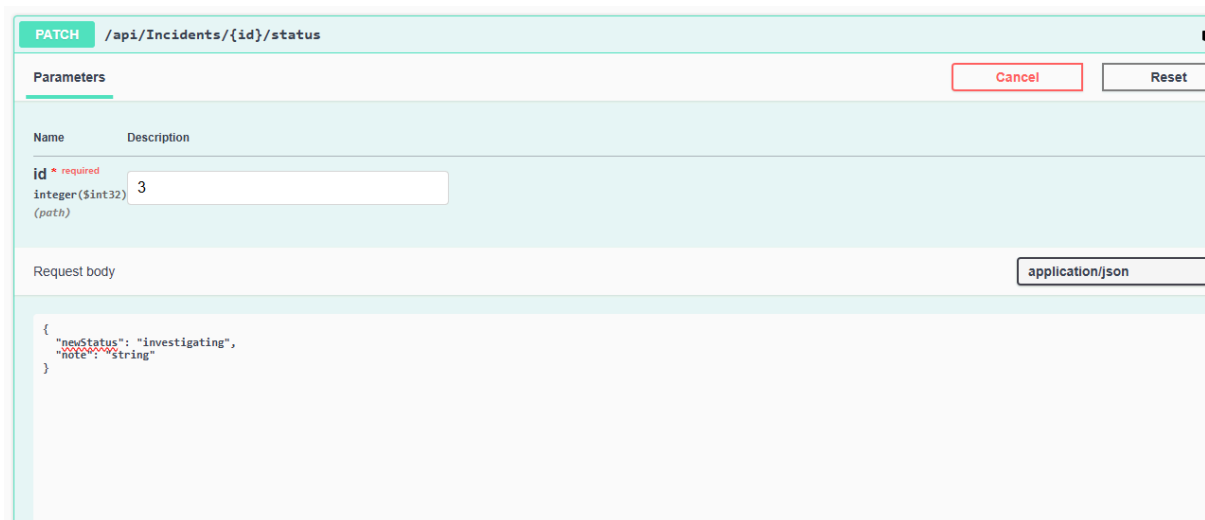
Authorization: *JWT required*

- Roles allowed: *Reviewer, Administrator*

Moves an incident to a new status, enforcing the workflow's legal transitions.

HOW IT WORKS

- IncidentService.AllowedTransitions defines exactly which moves are legal: Open→Investigating; Investigating→Resolved or Open; Resolved→Closed or Investigating; Closed is final.
- If the requested newStatus is not in the allowed list for the incident's current status, the service throws an InvalidOperationException, which the global exception middleware converts into a 400 Bad Request with a clear message — the illegal transition is rejected before any data changes.
- On success, updates Incident.Status and writes a new StatusHistory row recording the old status, new status, who changed it, when, and an optional note.



The image shows a screenshot of an API client interface for a PATCH request to the endpoint `/api/Incidents/{id}/status`. The interface includes a "Parameters" section with a table for path parameters, a "Request body" section with a JSON payload, and "Cancel" and "Reset" buttons.

Name	Description
id * required integer(\$int32) (path)	3

Request body

```
{  "newStatus": "investigating",  "note": "string"}
```

application/json

Code	Details
200	Response body <pre>{ "id": 3, "title": "Broken glass injury", "description": "bleeding and minor cuts leading to wounds or hidden shrads", "category": "Slip/Fall", "severity": "Medium", "status": "Investigating", "equipment": "Beaker#B6", "occurredAtUtc": "2026-09-01T14:20:29.958", "createdByName": "Riya Reviewer", "assignedToName": null, "createdAtUtc": "2026-09-01T14:26:48.4609029", "updatedAtUtc": "2026-09-01T14:33:28.03239132", "resolutionDetails": null, "investigationNotes": [], "correctiveActions": [] }</pre> Response headers <pre>content-type: application/json; charset=utf-8 date: Tue, 01 Sep 2026 14:33:27 GMT server: Kestrel transfer-encoding: chunked</pre>
Responses	
Code	Description
200	OK

Knowledge Base (Document) Endpoints

Controller: DocumentsController — Base route: /api/documents — These endpoints manage the documents the AI assistant is allowed to learn from, including the approval governance workflow.

Method	Route	Auth Req	Description
GET	/api/documents	Any logged-in user	List every document and its approval status
POST	/api/documents/upload	Reviewer, Administrator	Upload a .txt or .pdf candidate document
PATCH	/api/documents/{id}/approve	Administrator	Approve a pending document
PATCH	/api/documents/{id}/reject	Administrator	Reject a pending document
DELETE	/api/documents/{id}	Administrator	Permanently delete a document and its chunks

POST /api/documents/upload

Authorization: *JWT required* • Roles allowed: *Reviewer, Administrator*

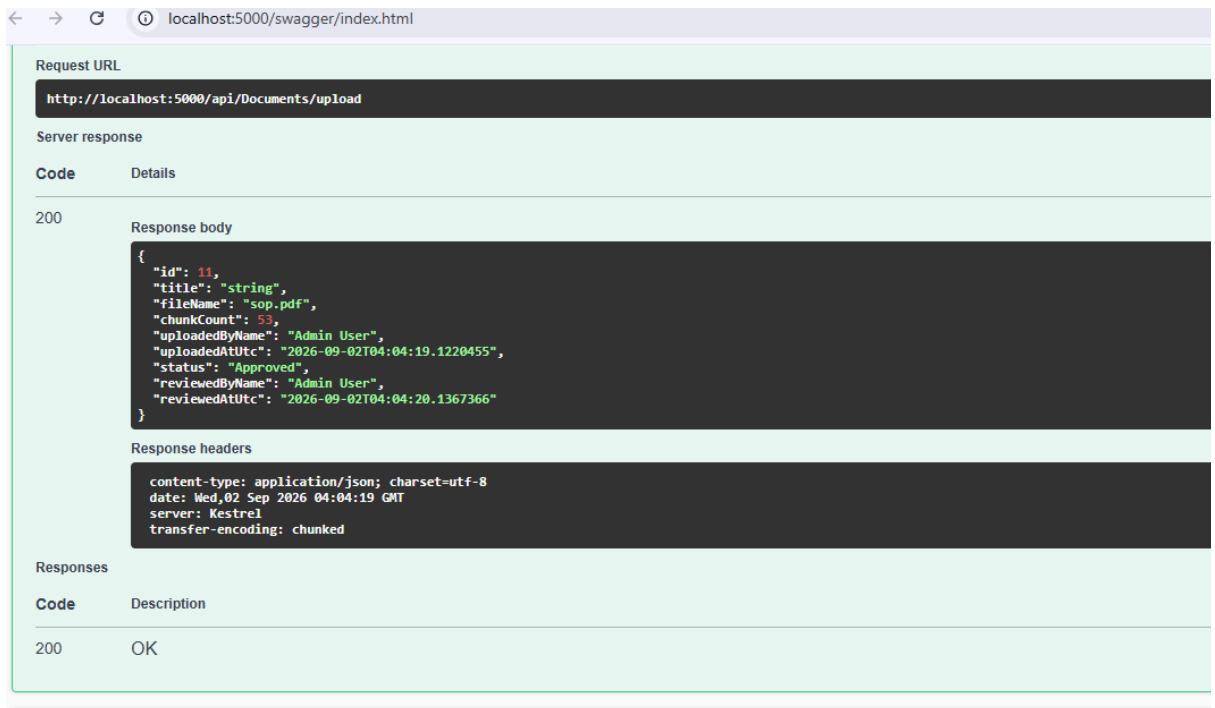
Uploads a knowledge document (.txt or .pdf), extracts its text, splits it into chunks, embeds each chunk, and stores everything permanently — but does not make it usable by the AI until approved.

HOW IT WORKS

- For .pdf files, text is extracted page-by-page using the PdfPig library (pure managed code, no external dependencies).
- The full text is split into ~120-word overlapping chunks (TextChunker.Chunk).
- Each chunk is converted into a numeric embedding vector (IEmbeddingService.EmbedAsync) and stored alongside the chunk text.
- The document is inserted with Status = Pending.
- Special case: if the uploader is an Administrator, the document is immediately auto-approved in the same request (ApproveAsync is called right after ingestion) — an Administrator is the approver, so there is no reason to require them to approve their own upload as a separate step.

The screenshot displays an API client interface for a POST request to the endpoint `/api/Documents/upload`. The interface is organized into several sections:

- Method and Endpoint:** A green bar at the top indicates the method is `POST` and the endpoint is `/api/Documents/upload`.
- Parameters:** A section below the endpoint bar shows "No parameters" and includes a red "Cancel" button.
- Request body:** A section with a "multipart/form" label. It contains two input fields:
 - file:** A text input with the value `string($binary)` and a "Choose File" button. The file name `sop.pdf` is visible. Below the input is a checkbox labeled "Send empty value".
 - title:** A text input with the value `string`. Below the input is a checkbox labeled "Send empty value".
- Buttons:** At the bottom, there is a blue "Execute" button and a "Clear" button.



PATCH /api/documents/{id}/approve

Authorization: *JWT required* • Roles allowed: *Administrator*

Approves a Pending document, making it — and only it, among that document's chunks — usable by the AI assistant's retrieval step for the first time.

HOW IT WORKS

- Sets `DocumentItem.Status = Approved`, records `ReviewedByUserId` and `ReviewedAtUtc`.
- This is the specific write that changes `AiController`'s behavior: `RagService`'s retrieval query filters strictly on `Status == Approved`, so a chunk belonging to this document is invisible to every AI query until this exact endpoint is called.

Request URL

http://localhost:5000/api/Documents/11/approve

Server response

Code

Details

200

Response body

```
{
  "id": 11,
  "title": "string",
  "fileName": "sop.pdf",
  "chunkCount": 53,
  "uploadedByName": "Admin User",
  "uploadedAtUtc": "2026-09-02T04:04:19.1220455",
  "status": "Approved",
  "reviewedByName": "Admin User",
  "reviewedAtUtc": "2026-09-02T04:06:52.7594907"
}
```

Response headers

```
content-type: application/json; charset=utf-8
date: Wed,02 Sep 2026 04:06:52 GMT
server: Kestrel
transfer-encoding: chunked
```

Responses

Code

Description

200

OK

AI Assistant RAG Endpoints

Controller: AiController — Base route: /api/ai — These two endpoints are the system's Retrieval-Augmented Generation feature: grounded question answering and AI-generated incident summaries

Method	Route	Auth Req	Description
POST	/api/ai/ask	Any logged in user	Ask a grounded question against approved documents

GET	/api/ai/incidents /{id}/summary	Any logged in user	AI summary + suggested next step for one incident
-----	------------------------------------	-----------------------	--

POST/api/ai/ask

Authorization: *JWT required*

The core RAG endpoint. Answers a free-text question using ONLY text retrieved from approved knowledge-base documents, and explicitly refuses to answer when nothing relevant is found.

HOW IT WORKS

- Step 1 (Filter): the question's keywords are compared against every APPROVED document chunk's keywords; only chunks sharing at least one real, exact word survive. This is a hard requirement, not a similarity score.
- Step 2 (Rank): only among chunks that passed Step 1, cosine similarity between the question's embedding and each chunk's embedding ranks the survivors best-first; the top K (default 4) are kept.
- Step 3 (Generate): the surviving chunks are turned into a short, bullet-point answer. If zero chunks survived Step 1, the response is the fixed string “Out of context - I don't have an approved knowledge-base document that answers this.” instead of any generated guess.
- Every call is logged to the AiQueryLog table (question, answer, which chunk ids were used, whether it was grounded) for auditability.
- The response's isGrounded flag and each source's similarityScore give a transparent, inspectable trail from question to answer.

Ai

POST

/api/Ai/ask

Parameters

Cancel

No parameters

Request body

applicatio

```
{
  "question": "what to do during chemical spill"
}
```

http://localhost:5000/api/Ai/ask

Server response

Code

Details

200

Response body

```
{
  "answer": "- back of the working surface because it will block airflow to the lower slot under the baffle in the back Separate and elevate equipment by u...\n- Chemical fume hoods must be kept clean and free from unnecessary items and debris at all times Solid material (paper, tissue, aluminum foil,...\n- Minimize the amount of bottles, beakers and equipment used and stored inside the hood because these items interfere with the airflow across...\n- Chemicals should not be stored in a hood because they will likely become involved if there is an accidental spill, fire or explosion in the...\n- become involved if there is an accidental spill, fire or explosion in the hood, thus creating a more serious problem.",
  "isGrounded": true,
  "sources": [
    {
      "documentId": 11,
      "documentTitle": "string",
      "chunkIndex": 6,
      "snippet": "back of the working surface because it will block airflow to the lower slot under the baffle in the back Separate and elevate equipment by using blocks or lab jacks to ensure that air can flow easily around and under the...",
      "similarityScore": 0.331
    },
    {
      "documentId": 11,
      "documentTitle": "string",
      "chunkIndex": 7,
      "snippet": "become involved if there is an accidental spill, fire or explosion in the hood, thus creating a more serious problem • Fume hoods are not flammable cabinets and do not offer fire protection for materials stored inside • ...",
      "similarityScore": 0.293
    },
    {
      "documentId": 11,
      "documentTitle": "string",
      "chunkIndex": 3,

```

Download

Response headers

```
content-type: application/json; charset=utf-8
date: Wed,02 Sep 2026 04:14:27 GMT
server: Kestrel
transfer-encoding: chunked
```

Administration Endpoints

Controller: AdminController — Base route: /api/admin — Every endpoint in this controller requires the Administrator role at the class level ([Authorize(Roles = "Administrator")]); a Reviewer or Lab User token is rejected with 403 Forbidden for all of them, with no exceptions.

METHOD	ROUTE	AUTH REQ	DESCRIPTION
GET	/api/admin/users	Administrator	List every user

			account
PATCH	/api/admin/users/{id}/role	Administrator	Change a user's role
PATCH	/api/admin/users/{id}/active	Administrator	Activate/deactivate a user
POST	/api/admin/users/{id}/reset password	Administrator	Issue a new temporary password